

Mateu como renderer sobre Oracle Visual Builder

Convertir una app VBCS/Redwood (Spectra) en un renderer que dialoga con la API de Mateu

Documento de diseño · 24 jul 2026 · POC ejecutable en `.dev/vb/renderer-poc/`

Objetivo: que una **app real de Visual Builder**, corriendo en su propio runtime, actúe como renderer de Mateu — aprovechando lo que VB ya hace bien (dynamic UI, shell Spectra, chains, temas, i18n) en vez de forzar componentes en un host ajeno. VB se comporta como “*un renderer más*” del mismo `UIIncrementDto`, coherente con la filosofía del repo (un solo modelo de wire, muchos frontends).

1 · Las tres apps VB de referencia

En `.dev/vb/` hay tres exportaciones VBCS que representan tres estilos de app Redwood. Son **referencia** (qué etiquetas `oj-*`, slots, eventos y shell usar), no el runtime.

App	Qué es	Para qué la usamos
<code>dashboard</code>	JET web-applayout clásico (nav-list + drawer), sin Spectra, JET 15.1.5.	Referencia del chrome JET a bajo nivel y de las <i>action chains</i> en JSON.
<code>empty</code>	Plantilla Redwood dinámica: <code>oj-dyn-form</code> / <code>oj-dynamic-table</code> + Business Objects + dynamicLayouts.	El motor dirigido por metadatos que reutilizamos para pintar forms/tablas.
<code>frontoffice</code>	Redwood Starter con el pack <code>oj-sp</code> (Spectra): shell, smart-search, dashboard, foldout, item-overview, welcome, guided-process...	La base sobre la que construir: shell auténtica + catálogo de plantillas.

El catálogo `oj-sp-*` del `frontoffice` es casi 1:1 con los arquetipos que Mateu ya emula (Smart Search, Collection Detail, General/Item Overview, Foldout, Welcome, Guided Process, Dashboard, Drawers, Command Center).

2 · Decisión de arquitectura

Bridge JS dentro de la app VB. La app VB consume el `UIIncrementDto` estándar y un módulo `MateuBridge` (+ action chains) lo traduce a UI de VB *en el navegador*. El backend de Mateu no se toca; VB es frontend puro.

Descartado: (a) emitir metadatos `oj-dynamic` desde el servidor (rompería el modelo de un solo wire y añade un target de backend); (b) hospedar Mateu dentro del runtime de chains de VB (perderíamos el bus de actualizaciones parciales por id, enrutado, drawers, dirty-guard... todo ya resuelto en el core neutral).

Las 4 tareas de un renderer ↔ primitiva nativa de VB

Tarea	Primitiva VB
Transporte (llamar a la API)	Service Connection / <code>fetch</code> + una action chain <code>callMateu(route, actionId, state) → increment (POST /{base}/mateu/v3/components/_/action)</code> .
Pintar desde metadatos	<code>oj-dyn-form</code> / <code>oj-dynamic-table</code> (provider + <code>value</code> two-way) o un dispatcher por tipo.
Capturar eventos	eventListeners → chains; <code>value</code> two-way ya recoge el estado.
Aplicar la respuesta	variables/ADP + chains: reasignar → repintar; comandos → navegar/toast/drawer.

3 · El motor que reutilizamos: `oj-dyn-form`

VB ya trae un renderer dirigido por metadatos. Encaja 1:1 con Mateu:

```
<oj-dyn-form
  metadata="[ [ $variables.mateuProvider ] ]"      <!-- QUÉ pintar (provider desde JSON) -->
  layout="default"
  value="{{ $variables.componentState }}"          <!-- el ESTADO (object, two-way) =
componentState -->
</oj-dyn-form>
```

- `metadata` = un *metadata provider* construido en JS desde el árbol de Mateu (`JsonMetadataProvider`).
- `value` two-way = el `componentState` de Mateu; se actualiza solo al teclear.
- La estructura `layouts → displayProperties` está confirmada por `.dev/vb/empty/dynamicLayouts/Persona` .

4 · Bootstrap: primera llamada al cargar (`vbEnter`)

“Al cargar la app” = el evento `vbEnter` del app-flow → una chain → una función JS que hace el `fetch` .

```
// app-flow.json (fusionar)
"eventListeners": { "vbEnter": { "chains": [ { "chain": "loadInitialScreen" } ] } },
"variables": { "mateuIncrement": { "type": "object" } },
"constants": { "mateuBaseUrl": { "type": "string", "defaultValue": "http://localhost:8080" },
               "mateuRoute": { "type": "string", "defaultValue": "" } }
```

```
// app-flow.js – el bridge, versión mínima
async callMateu(baseUrl, route, actionId, componentState) {
  const res = await fetch(`${baseUrl}/mateu/v3/components/_/action`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ route, actionId, componentState: componentState || {} })
  });
  if (!res.ok) throw new Error(`Mateu ${res.status}`);
  return await res.json();      // UIIncrementDto
}
```

Gotchas: `actionId` de carga = `__load__` ; CORS entre el host de VB y Mateu (habilitar en el backend o servir same-origin); en producción, sustituir `fetch` por una Service Connection.

5 · Pintar dinámico: el switch de campos

El bridge decide el widget (JS, testeable) y anota `_widget` ; la página solo tiene ramas `oj-bind-if` .

```

widgetFor(f) {
  const st = (f.stereotype||'').toLowerCase(), dt = (f.dataType||'').toLowerCase();
  if (f.readOnly || ['plaintext','badge','money','bulletedlist'].includes(st)) return 'readonly';
  if (f.options?.length || ['multiselect','treeselect'].includes(st)) return 'select';
  if (['textarea','richtext'].includes(st) || f.multiline) return 'textarea';
  if (dt.includes('bool')) return 'boolean';
  if (dt.includes('datetime')) return 'datetime';
  if (dt.includes('date')) return 'date';
  if (['integer','long','number','double','decimal','money'].some(x=>dt.includes(x))) return
'number';
  return 'text';
}

```

```

<oj-bind-for-each data="[ [ $variables.fields ] ]"><template>
  <oj-bind-if test="[ [ $current.data._widget === 'number' ] ]">
    <oj-c-input-number label-hint="[ [ $current.data.label ] ]"
      value="{{ $variables.componentState[$current.data.fieldId] }}"></oj-c-input-number>
  </oj-bind-if>
  <!-- ramas text / boolean / date / select / textarea / readonly -->
</template></oj-bind-for-each>

```

6 · `applyIncrement` : plan puro + efectos

JS puro calcula un “plan” (sin efectos secundarios); una action chain lo aplica (asignar variables, navegar, toast, cerrar drawer). VB posee la navegación/variables/eventos — meter eso en JS suelto es forzarlo.

El `UIIncrementDto`: { `messages`, `commands`, `fragments`, `banners`, `componentState` }. Fragmentos con `action` ∈ { `Add`, `Replace`, `ReplaceKeepData` } y `targetComponentId`; comandos con `type` / `data` / `targetComponentId` (`NavigateTo`, `SetWindowTitle`, `CloseModal`, `MarkAsClean/Dirty`, `DownloadFile`, `RunAction`, `DispatchEvent` ...).

La chain (pipeline lineal de `ifAction`s)

```
computePlan → applyScalars → setTitle
  → ifBanners -true→ pushBanners ┘
    └false→ ifToast -true→ fireToast ┘
      └false→ ifNav
        ifNav -true→ ifNavRoute -true→ navInternal (loadRoute) ┘
          └false→ navExternal (window.open)┘
            └false→ ifClose
              ifClose -true→ closeDrawer ┘
                └false→ ifDownload -true→ doDownload ┘
                  └false→ ifRun
                    ifRun -true→ runNext (encadena el siguiente increment) ┘
                      └false→ done
```

Builtins confirmados en `.dev/vb/dashboard`: `assignVariablesAction` ({`target`: {`source`} }), `ifAction` (condition + outcomes.true/false), `callModuleFunctionAction` (module/functionName/params, resultado en `$chain.results.<id>`), `navigateAction`, `fireDataProviderEventAction` ({`target`,`add`: {`data`} }). A verificar en Designer: clave de `callChainAction` (`id` vs `chain`), `fireEventAction` (name/payload; el shell del frontoffice ya escucha `spShowToast`).

7 · Cerrar el bucle: `loadRoute` / `runActionChain` / `onAction`

No son tres cosas: es **un transporte** parametrizado; todo desemboca en `applyIncrement` .

Chain	route	actionId	state que envía
<code>loadRoute</code>	la nueva	<code>__load__</code>	{ }
<code>runActionChain</code>	la del contexto	el del botón	<code>state</code> del contexto (+ params extra)
<code>onAction</code>	—	dispara <code>runActionChain</code> con el <code>actionId</code> del botón	—

Los botones también salen del increment (`collectActions`); cada uno dispara `onAction(actionId, contextId)` . Como `componentState` es two-way, guardar es “manda el estado que ya tienes”. Tras guardar, Mateu suele devolver `[message, markAsClean]` sin form: el ternario en `applyScalars` conserva las variables y no borra la pantalla.

8 · El unificador: registro `targetComponentId` → `contexto`

Host, drawer e isla son lo mismo: una “superficie de render” con estado propio, direccionada por su `targetComponentId`. El `increment` es una lista de operaciones direccionadas. `applyIncrement` ⇒ un **reducer puro** sobre el árbol de superficies + una **cola de efectos**. Es el mismo patrón del bus RxJS de Mateu (`ConnectedElement`), reimplementado por id.

El registro son dos variables de app: `contexts` (mapa `id-contexto`, `host = __root__`) y `stack` (ids de drawers abiertos). Más una `shell` cuando llega un `App`.

```
context = { id, kind:'host'|'drawer'|'island', route, pageType, pageWidth,
            tree, state, dirty, /* título/posición/size si drawer */ }
```

El reducer (núcleo del POC). Enruta fragmentos por id y traduce comandos a efectos; algunos mutan el registro:

```
export function reduceContexts(reg, increment) {
  const contexts = { ...reg.contexts }; const stack = [...reg.stack]; let shell = reg.shell || null;
  const effects = { toasts:[], banners: increment.banners||[], navigate:null, download:null, runActions:[], docTitle:null };
  for (const m of increment.messages||[]) effects.toasts.push({ text:m.text||m.title, variant:m.variant||'info' });

  for (const fr of increment.fragments||[]) {
    const md = fr.component?.metadata || fr.component || {};
    if (md.type === 'App') { shell = { title:md.title, menu:md.menu||[], variant:md.variant, appContext:md.contextSelectors||[], headerActions:md.contextActions||[], width:md.pageWidth }; continue; }
    if (fr.action === 'Add') { const ctx = buildOverlay(fr); contexts[ctx.id]=ctx;
      if(!stack.includes(ctx.id)) stack.push(ctx.id); continue; }
    const id = fr.targetComponentId || '__root__';
    const prev = contexts[id] || { id, kind: id==='__root__'?'host':'island', state:{} };
    contexts[id] = { ...prev, kind: prev.kind || (id==='__root__'?'host':'island'),
      tree: fr.component || prev.tree, pageType: md.pageType ?? prev.pageType, pageWidth: md.pageWidth ?? prev.pageWidth,
      state: fr.action==='ReplaceKeepData' ? { ...prev.state, ...(fr.state||{}) } : (increment.componentState || fr.state?.[fr.component?.id] || fr.data || prev.state), dirty:false };
  }

  for (const c of increment.commands||[]) { const t = c.targetComponentId;
    switch (c.type) {
      case 'SetWindowTitle': effects.docTitle = c.data; break;
      case 'NavigateTo': { const d=String(c.data||''); effects.navigate = /^https?:/.test(d)? {url:d}:{route:d}; break; }
      case 'CloseModal': { const id = (t&&contexts[t])?t:stack[stack.length-1];
        if(id){ delete contexts[id]; const i=stack.indexOf(id); if(i>=0) stack.splice(i,1); }
        break; }
      case 'MarkAsClean': if(contexts[t]) contexts[t]={...contexts[t],dirty:false}; break;
      case 'MarkAsDirty': if(contexts[t]) contexts[t]={...contexts[t],dirty:true}; break;
      case 'DownloadFile': effects.download = c.data; break;
      case 'RunAction': effects.runActions.push(c.data); break;
    }
  }
  return { contexts, stack, shell, effects };
}
```

`applyIncrement` colapsa a: `reduce` → asignar `contexts/stack` → pipeline de efectos. Cerrar un drawer deja de ser una rama: es puro estado (`delete` + `splice`). Lo saliente: `onAction(actionId, contextId)` usa `contexts[contextId].route/state`.

9 · Altura: app completa, página rica y formulario

El registro y el reducer **no cambian con la altura**. Lo que cambia: un contexto guarda el **árbol** (`tree`), no campos planos; y el render pasa a ser un **dispatcher recursivo por tipo**. Un `App` no es contenido: configura la **shell**.

Dispatcher recursivo (el análogo VB de `renderClientSideComponent`)

```
<!-- fragments/mateu-node : recibe { node }; recurre por sí mismo -->
<oj-bind-if test="[[ node.metadata.type === 'Card' ]]">
  <oj-sp-card header-title="[[ node.metadata.title ]]">
    <oj-bind-for-each data="[[ node.children ]]"><template>
      <oj-vb-fragment name="mateu-node">
        <oj-vb-fragment-param name="node" value="[[ $current.data ]]"></oj-vb-fragment-
param>
        </oj-vb-fragment>
      </template></oj-bind-for-each>
    </oj-sp-card>
  </oj-bind-if>
```

Mapa tipo Mateu → componente Oracle

Tipo Mateu	Oracle
App	<code>oj-sp-simple-ui-shell</code> + <code>oj-sp-global-header</code> + <code>oj-sp-navigator</code>
Layouts / Div / Container	<code>oj-flex</code>
FormLayout / FormField	<code>oj-dyn-form</code> o el switch de widgets
FormSection / Card / TabLayout	<code>oj-sp-section</code> / <code>oj-sp-card</code> / <code>oj-c-tab-bar</code>
DashboardLayout / Scoreboard / MetricCard / DashboardPanel	<code>oj-sp-dashboard-grid</code> / <code>-scoreboard</code> / <code>-metric-card</code> / <code>-panel</code>
FoldoutLayout / FoldoutPanel	<code>oj-sp-foldout-layout</code> / <code>-panel</code>
HeroSection / EntityHeader	<code>oj-sp-welcome-page</code> / <code>oj-sp-header-general-overview</code>
TaskQueue / Crud / Table / Grid	<code>oj-sp-task-card</code> lista / <code>oj-dynamic-table</code>

Clave: la mayoría de arquetipos son *composición* de un núcleo de ~20 tipos. `item-overview` = `HorizontalLayout(Card + TabLayout)`; `general-overview` = `EntityHeader + Cards`; `collection-detail` = `TaskQueue + detalle`; `welcome` = `HeroSection + DashboardLayout`. Cubre ese núcleo y **salen gratis**. Sólo unos pocos (`DashboardLayout`, `FoldoutLayout`, `HeroSection`, `EntityHeader`, `Scoreboard`, `MetricCard`) tienen tipo de wire dedicado. `pageType` / `pageWidth` (ya en el wire) sólo eligen la plantilla exterior y el ancho.

App = shell. Una rama del reducer fija `shell = {title, menu-navigator, variant, appContext, headerActions, width}`; el contenido enrutado llega como fragmento aparte dirigido al host. Federación (menús remotos con `baseUrl`) = paso avanzado: cada opción remota es a su vez un `App` que se busca y se fusiona.

10 · POC ejecutable y validación

En `.dev/vb/rendererer-poc/` : `reduceContexts.mjs` (JS puro, el mismo código que serían métodos de `app-flow.js`) + `fixtures/*.json` (increments fieles a los DTOs) + `test.mjs` (`node:assert` , sin framework).

```
$ cd .dev/vb/rendererer-poc && node test.mjs
✓ carga inicial registra el host con tree; los campos se derivan del árbol
✓ item-overview se registra IGUAL que un form (host + tree), pageType=detail
✓ foldout se registra IGUAL (host + tree), pageType=detail, pageWidth=edgeToEdge
✓ form / item-overview / foldout comparten forma de contexto (unificación)
✓ App configura la shell y no crea contexto de contenido
✓ App y luego una página conviven: shell + contexto host
✓ Add crea un drawer con su tree y lo apila sin tocar el host
✓ guardar-en-drawer cierra el drawer y emite toast + runAction
✓ isla por id: State fusiona conservando el árbol; host intacto
✓ NavigateTo interno produce efecto route sin tocar el registro
10 tests OK
```

El reducer + registro + efectos queda validado de punta a punta, incluidos los tres casos difíciles (drawer apilado, cierre por estado, isla por id) y la unificación de altura (form / item-overview / foldout / app).

Siguiente paso — fixtures reales. Un `capture.mjs` de 15 líneas que hace `POST` a un backend Mateu vivo (`__load__` , un `save` , un `editInDrawer` , un `master-detail`) y vuelca la respuesta a `fixtures/` . Los mismos asserts corren contra ellos y afinan los paths defensivos del reducer sin abrir VB.

11 · Orden de construcción y frontera honesta

1. **A** Transporte + `loadRoute` / `runActionChain` / `onAction` → CRUD plano (el 80%).
2. **B1** Un drawer (fragmentos `Add`) → CRUD-en-drawer.
3. **B2** Pila de overlays + islas vía el registro por id → master-detail embebido.
4. **C** Dispatcher recursivo `mateu-node` + mapa tipo → oj-sp → páginas ricas (dashboard, foldout, overviews).

Cada paso es ejecutable por sí solo y ninguno obliga a rehacer el anterior (el registro por id de B2 generaliza el contexto único).

Frontera honesta: el modelo cubre la topología real de Mateu (1 host + pila de drawers + islas con nombre); la colocación arbitraria “cualquier componente en cualquier sitio” es lo único que VB no da gratis, y Mateu no la pide. Puntos a confirmar en el Designer: construcción exacta de `JsonMetadataProvider` (versión oj-dynamic), clave de `callChainAction` , params de `fireEventAction` , y los paths de wire reales (`component.id` , `initialData` , metadata del `Drawer`) contra un increment capturado.